

Language Models in Practice

Errin W. Fulp
fulp@wfu.edu

Fall 2026. CSC 191, Language Models in Practice.

Lecture: To be determined. (Meets once weekly for 90 minutes.)

TEXT, NOTES, AND HANDOUTS

- There is **no required textbook**. Short pre-reads, handouts, and reference materials will be provided for each session via the Wake Forest Canvas site <https://canvas.wfu.edu>

COURSE DESCRIPTION AND OBJECTIVES

An interdisciplinary introduction to solving real-world problems using modern language models. Moving beyond a basic survey of the technology, this course focuses on building technical judgment, establishing an accurate mental model of model behaviors, and defining what these systems can and cannot do. Students will learn the practical craft of designing, testing, and implementing reliable systems and user interactions powered by generative AI.

A central objective of the course is developing the critical judgment required to determine which approach, or combination thereof, is best suited to address a given problem. Neither paradigm is treated as the lesser option; understanding their trade-offs is fundamental to the course. By the conclusion of this course, students will be able to:

1. Explain Core Model Mechanics: Explain, at a conceptual level, how language models process information (including next-token prediction, the distinction between training and inference, and context window mechanics) and diagnose common failure modes such as hallucinations, recency bias, and exactness limitations.
2. Assess Task Suitability: Classify organizational and technical challenges using a suitability framework to objectively evaluate when a language model is, and is not, the appropriate tool for a given task.

3. Engineer Precise Interactions: Design precise, structured prompts and engineered context to transform ambiguous queries into reliable, repeatable model interactions.
4. Implement Output Verification: Evaluate and verify model outputs to distinguish trustworthy responses from plausible inaccuracies, while systematically accounting for system limitations, data privacy, and algorithmic bias.
5. Construct Multi-Step Workflows: Design and implement multi-step workflows and data-grounded, persistent assistants to automate complex, recurring tasks.
6. Establish Rigorous Evaluation Protocols: Evaluate model-based systems systematically, progressing from manual, rubric-based matrix assessments to automated test harnesses.
7. Architect Agentic Solutions: Analyze the technical landscape of modern AI integration (including APIs, tool use, function calling, orchestration, and the Model Context Protocol [MCP]) to reason about system architecture, agentic problem-solving patterns, and the appropriate level of system autonomy.

REQUIREMENTS AND EXPECTATIONS

There is **no programming prerequisite**. What the course requires instead is an interest in the material, a willingness to experiment and iterate, an enthusiastic attitude toward participation, and a respectful attitude toward your peers. Because this is a small, project-driven course that meets only once a week, attendance and active participation are essential: each meeting is a checkpoint, and the real learning happens in the practice you do between sessions. I expect you to attend each meeting, complete the pre-reads and between-session work in advance, and work actively during in-class time. Your performance will be assessed using in-class worksheets, homework, examinations, and your semester project.

Examinations

Two examinations will be given. A **mid-term examination** is administered in class near the middle of the term, and a **final examination** at the University-assigned time and date for this course. Both examinations are closed-book and

are completed **without the assistance of a language model or other outside resources**. Make-up examinations will be given only for a **University excused absence**.

In-Class Worksheets

Most sessions include structured hands-on activities, group work, and short worksheets completed during class, decomposing a workflow, fixing a prompt, drafting a rubric, or critiquing a proposal. These are graded for participation and completion. Because they happen during class, **no** make-up worksheets will be given except for a **University excused absence**.

Homework

Homework is assigned weekly and is where the course’s real learning consolidates. **No** late homework will be accepted unless due to a **University excused absence**. Some homework is *differentiated*: students working at the code level and students working at the behavior/design level are accountable for different layers of the same task (see *Academic Integrity*).

Projects

Each student will develop a **personal project** of their own choosing, with guidance and subject to approval from the instructor. Deliverables include a written proposal, weekly updates, a structured peer critique, and a final presentation.

Tools and Access

Coursework will use a university-approved laptop together with the language-model tools specified for each activity. The specific accounts, model access, and any local software you need will be provided or announced by the instructor. You are responsible for maintaining your accounts and access.

Grading

In-class worksheets	10%	Grade scale: Passing (100-70) and Non-Passing (69-0)
Homework	10%	
Mid-term exam	20%	
Project	40%	
Final exam	20%	

ACADEMIC INTEGRITY AND RESPONSIBILITY

All examinations, worksheets, homework assignments, and project deliverables must represent the student's own individual understanding. Unless an assignment explicitly authorizes peer collaboration, all work must be executed independently. Copying partial or complete solutions from another student, or submitting work one cannot independently explain or defend, constitutes an integrity violation and will be referred immediately to the University Judicial System.

Because the functional application of language models is the core academic subject of this course, their utilization is expected rather than prohibited, subject to the operational boundaries outlined in the following section. The overarching standard that applies across all deliverables is **accountability**: students remain the sole author of everything submitted, meaning they must completely understand, explain, and defend their work independently of any generation tool used to construct it.

To reinforce this standard, students will maintain an active log of model usage throughout the semester, detailing prompts utilized, variations returned, and how outputs were iteratively refactored. This log serves both as an academic learning journal and formal validation of authorship; if a question regarding authorship emerges, a student may be asked to produce this log alongside chronological development baselines tracing the project's evolution.

Assessment in this course distinguishes two layers of engineering responsibility. *Correctness of behavior*—evaluating whether a system handles execution properly for its intended deployment context—requires comprehensive domain understanding rather than code optimization; every student is held fully accountable for this dimension. *Correctness of implementation*—evaluating whether underlying programmatic code is structured correctly—applies solely to deliverables a student explicitly elects to execute at the programmatic code layer. Reading or writing low-level software code is not required to meet baseline course criteria; the applied lens enables students to satisfy all project criteria by architecting and evaluating system-level behavioral dynamics. When an assignment tracking parameter exceeds what a student can genuinely account for, the expected response is to seek instructional assistance or narrow the functional scope of the submission.

GENERATIVE AI

Generative AI is the subject of this course; that does not remove the obligations that accompany its use. Whenever you use a language model in your

coursework, the following guidelines apply:

- **Transparency.** Clearly indicate when and how you used a model, for example, whether it assisted with brainstorming, drafting, generating code, or checking your work. The use log described above supports this.
- **Verification.** Treat model output as a claim to be verified rather than an answer to be trusted. Where correctness matters, confirm it against authoritative sources, against your own evaluation criteria, or by re-deriving the result, and be prepared to demonstrate how you did so.
- **Accountability.** You must be able to explain and defend any work you submit independently of the tool that produced it. Work you cannot account for should not be submitted.
- **Data and privacy.** Do not provide personal, proprietary, or FERPA-protected information to a model. As your projects come to involve other people's information, you are additionally responsible for obtaining appropriate consent and for disclosing to users, honestly and clearly, that they are interacting with an AI system and what its limitations are.
- **Understanding.** These assignments are designed to develop your judgment. Using a model to produce work you do not understand defeats that purpose and will undermine your performance on the work that depends on it.

The objective is to use these tools to extend your judgment, not to substitute for it.

DISABILITIES AND SPECIAL ACCOMMODATIONS

If you have a disability that may require an accommodation for taking this course, then please contact the WFU Center for Learning, Access, and Student Success (CLASS, 758-5929) and request that the CLASS send me official notification of your accommodation needs as soon as possible. Please plan to meet with me to discuss accommodations and how my course requirements and activities may affect your ability to fully participate.

GRIEVANCE PROCEDURES

For complaints in the academic (i.e., classroom) setting, the student should talk personally with or send a written complaint explaining the concern directly

to the instructor. Should the student and instructor be unable to resolve the conflict, the student may then turn to the chair of the involved department for assistance. The chair will communicate with both parties, seek to understand their individual perspectives, and within a reasonable time, reach a conclusion and share it with both parties. If the student's complaint is not resolved by these procedures, the student should consult with the Office of Academic Advising for assistance. The Associate Dean for Academic Advising will consult with the parties to obtain a resolution. Finally, a student may appeal to the Committee on Academic Affairs which will study the matter, taking input from all parties, and reach a final decision concerning resolution. <https://bulletin.wfu.edu/undergraduate/wake-forest-college/student-complaints/>

CAMPUS CLOSINGS

In the event that the Wake Forest campus is closed and we are unable to have our regularly-scheduled class meetings, then contact will be maintained hopefully via Canvas and campus email. As a result, you should regularly consult email and Canvas for class updates. If these services are not available, then contact will be made via telephone and/or postal mail. We will continue with assignments in these situations, so always have your computer and course notes available, and we will maintain communication through the Internet, email, and/or postal mail.

COURSE SCHEDULE (15 WEEKLY SESSIONS)

The following is the tentative schedule for this course. The course meets once weekly; dates are given by the week in which the session falls. Topics and dates may change during the semester.

WEEK	SESSION	CONCEPTS
Aug 24	What Language Models Actually Are, and Aren't	How models predict text and why they fail; the four-category suitability map
Aug 31	Context Engineering and Precision Prompting	Prompting as technical writing; roles, structure, examples, iteration
Sep 7	Verification and Limits: Developing Taste	Calibration and verification; where models break; self-critique by hand
Sep 14	Designing Repeatable Workflows	Breaking a task into steps; plan-and-execute; chaining and routing
Sep 21	Bringing in Data	Retrieval vs. synthesis; grounding an assistant in curated documents
Sep 28	Evaluation Without Code: The Rubric Matrix	Why “vibe checks” fail; a rubric matrix that tests the hard cases
Oct 5	The API and the Programmatic Call Mid-term exam	What an API is; the request/response of a model call; token cost
Oct 12	Tool Use and Function Calling	Giving a model tools; the ReAct reason-act-observe loop
Oct 19	AI-Assisted Development	Building and debugging with an AI assistant; reviewing what it writes
Oct 26	Agent Safety: Budgets, Loops, and Knowing When to Stop	How agent loops fail; budgets, limits, and checkpoints
Nov 2	System Architecture and Orchestration	Components of a solution; workflows-to-agents spectrum; least autonomy
Nov 9	Structured Tool Access: MCP (<i>spring proposal launched</i>)	MCP as a standard tool-access layer vs. ad hoc function calling
Nov 16	Rigorous Evaluation: From Rubric to Harness	Automating the rubric as a test harness; regression testing
Nov 23	<i>Thanksgiving Break — no class</i>	
Nov 30	Real-World Readiness: Choosing a Model, Privacy, and Security	Frontier vs. local model selection; privacy, security, and honest disclosure
Dec 7	Capstone Presentations and Spring Launch	Project presentations and proposal critique; reflection; spring launch
<i>Finals</i>	Final Exam	